Hospital Management System

Project Report

1. Student Details

Name	Utkarsh Kumar Shakya
Roll number	24f1000016
Email	24f1000016@ds.study.iitm.ac.in
Course / Term	MAD1 Sep 2025

2. Project Details

Problem statement. Hospitals often manage patients, doctors, appointments and treatments through manual registers or disconnected software, which makes scheduling conflicts likely and patient history hard to track. The task is to build a role-based Hospital Management System (HMS) web application for Admins, Doctors and Patients using Flask, Jinja2/HTML/CSS/Bootstrap and SQLite.

Approach. The problem was split by role and by data. First the database was designed (users, departments, doctors, patients, availability, appointments, treatments), then shared business logic (slot availability, booking, rescheduling) was written once and reused by both the web pages and the API, and finally one Flask blueprint was built per role.

- **Admin:** pre-seeded programmatically after database creation (no registration). Dashboard shows total doctors, patients and appointments with Chart.js charts. Admin can add, edit, delete and blacklist doctors and patients, search doctors by name/specialization and patients by name, ID or phone, view all upcoming and past appointments, and manage departments.
- **Doctor:** dashboard with today's and this week's appointments and the list of assigned patients; can mark visits Completed or Cancelled, record diagnosis, prescription and notes, view full patient history, and set availability for the next 7 days.
- **Patient:** self-registration and login, profile editing, department list, doctor search by name, specialization and date, 7-day availability on doctor profiles, booking, rescheduling and cancelling, and an appointment history showing diagnosis and prescriptions.

Preventing double booking. A slot is validated in code before booking, and a partial unique index on (doctor_id, date, time) for Booked and Completed appointments makes the database itself reject a second booking. Slots in the past or not offered by the doctor cannot be booked.

Other features. Chart.js dashboard charts, JSON REST API, HTML5 plus server-side validation, hashed passwords, role-based access control (403 for wrong role), blacklisting that blocks login and cancels future appointments, and a responsive Bootstrap interface.

3. AI / LLM Declaration

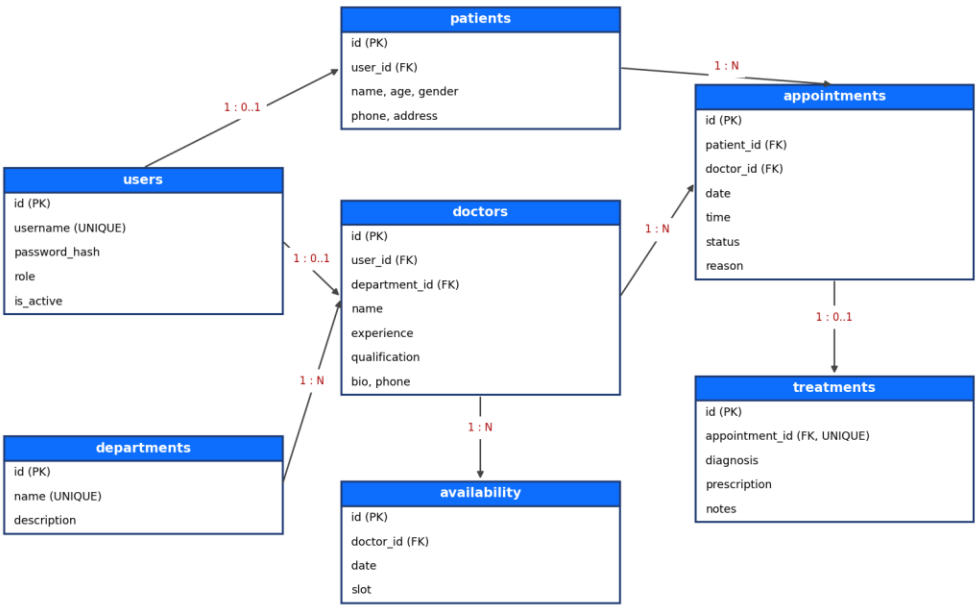
Gemini was used exclusively as a reference tool and to clear technical doubts regarding the framework during the course of this project. All application implementation details—including Flask routes, database schema and seeding code, Jinja2 templates, styling, the ER diagram, and this report—were fully designed, written, and developed independently

4. Frameworks and Libraries Used

Component	Technology
Back-end	Flask 3 (Python), sqlite3 module, Werkzeug (password hashing)
Templating / front-end	Jinja2, HTML5, CSS, Bootstrap 5.3 (CDN)
Charts	Chart.js 4 (CDN)
Database	SQLite (created programmatically in db.py)

Authentication	Flask sessions with role-based decorators

5. ER Diagram



users 1:0..1 doctors / patients; departments 1:N doctors; doctors 1:N availability; doctors 1:N appointments; patients 1:N appointments; appointments 1:0..1 treatments. Deleting a user cascades to the profile and its appointments.

6. API Resource Endpoints

All endpoints return JSON and require a logged-in session.

Method	Endpoint	Description / access
GET	/api/departments	List departments (any user)
GET	/api/doctors?q=	Search doctors by name or specialization (any user)
GET	/api/doctors/<id>	Doctor details with 7-day availability (any user)
POST	/api/doctors	Create doctor (admin)
PUT	/api/doctors/<id>	Update doctor (admin)
DELETE	/api/doctors/<id>	Delete doctor (admin)
GET	/api/appointments	Appointments scoped to the logged-in role
POST	/api/appointments	Book an appointment (patient)
PATCH	/api/appointments/<id>	Update status: Completed/Cancelled (doctor, admin); cancel only (patient)

7. Presentation Video

[Paste your Google Drive video link here (access: anyone with the link)]